Aldel Education Trust's

St. John College of Engineering and Management, Palghar

NAAC Accredited with Grade A+

DEPARTMENT OF COMPUTER ENGINEERING

LLM-Based Log Anomaly Detector

Harsh Bangera (Roll No. 3)
Manas Chaudhari (Roll No. 17)
Krish Mendon (Roll No. 62)
Anandan Nair (Roll No. 68)

Prof. Darshan Mhapsekar
31 August 2026

Contents

- Background of the Problem
- Problem Statement
- Motivation
- Literature Review
- Research Gaps
- Project Objectives
- Proposed Methodology
- System Architecture / Flow Diagram
- Implementation and Results
- Working System (screenshots)
- Technologies and Resources
- Project Schedule
- References

Background of the Problem

- Every server writes a log line for each login, process and network event; a small company produces millions a day and nobody reads them by hand.
- Security teams rely on a SIEM (Splunk, QRadar, Wazuh) that matches those lines against written rules.
- A rule matches text, not meaning: “Failed password for root” matches, the same event worded differently does not.
- Published surveys of log anomaly detection [R2, R7] evaluate on fixed datasets where the attacker is not trying to avoid the detector.

Background — What We Measured

- Sigma rules: 0 of ~3,300 match an SSH authentication line (searched for Failed password, Accepted password, Invalid user, pam_unix).
  - No coverage at all for the commonest attack surface on Linux.
- Wazuh caught 91% of a brute force’s lines — and 29% of the same attack reworded. Same behaviour, different words.
- Our detector: 97–100% of those lines in both conditions.
- Alert volume on one clean corpus: Wazuh 4,546 alerts, ours 1,415 incidents, for 9 real attacks — alert fatigue, measured on both sides.
  - All four measured by us on real OpenSSH logs; sources in the report.

Problem Statement

- SIEM rules match the text of a log line, not its meaning, so an attacker who rewords an attack walks straight past them.
- And nobody reports how much detection is actually lost when the attacker is deliberately hiding, so the size of the problem is unknown.

Motivation

- Analysts drown in alerts: a production SIEM raised 4,546 alerts for 9 real attacks on one clean corpus.
- Words change, behaviour does not — many failures from one place, or a login from two countries an hour apart, survive rewording.
- Semantic detection can follow that; string matching cannot.
- It has to run where the budget is: the whole system runs on an ordinary laptop, no GPU and no cloud account.
  - A GPU only speeds the written explanation up: 3.9 s versus 18.2 s per incident.

Literature Review (1 of 2)

Journal papers only — IEEE Transactions and Web of Science / Scopus indexed journals. Full PDFs are attached with this presentation.

Paper (journal, year)	**Method**	**Finding**	**Limitation**
R1 — Almodovar et al., LogFiT IEEE Trans. Network & Service Mgmt, 2024	BERT fine-tuned on log text, self-supervised	Language models read logs without labelled data	Clean benchmark logs only; no evading attacker
R2 — Landauer et al., survey Machine Learning with Applications, 2023	Survey of deep learning for log anomaly detection	Maps every major method and dataset in the field	Reports evaluation is almost always static and non-adversarial
R3 — Ali et al. Empirical Software Engineering, 2025	Classical ML versus deep learning, measured	Simpler models compete with deep models on logs	Clean data only; no adapting attacker
R4 — Wu, Li & Khomh Empirical Software Engineering, 2023	Compares ways of turning logs into vectors	Representation choice changes results a lot; embeddings work well	Does not test reworded or disguised attacks

Literature Review (2 of 2)

All eight evaluate on data where the attacker is not trying to evade the detector being tested.

Paper (journal, year)	Method	Finding	Limitation
R5 — Khan et al. Empirical Software Engineering, 2024	Measures how parsing quality affects detection	Poor parsing quietly ruins detection accuracy	Parsing studied on clean logs
R6 — Pang et al., review ACM Computing Surveys, 2021	Field review of deep anomaly detection	Standard definitions and method families	General; not security-log specific
R7 — Kheddar, survey Information Fusion, 2025	Transformers and LLMs for intrusion detection	LLMs are moving into intrusion detection quickly	Evaluation still on fixed public datasets
R8 — Debicha et al., TAD Future Generation Computer Systems, 2023	Attackers actively evading a detector	Evasion measurably breaks detectors	Network traffic, not logs; no SIEM comparison

Research Gaps

Our gap: how much worse does each system get when the attacker tries to hide — measured on the same logs, at the same moment, for a research detector and a production SIEM together.

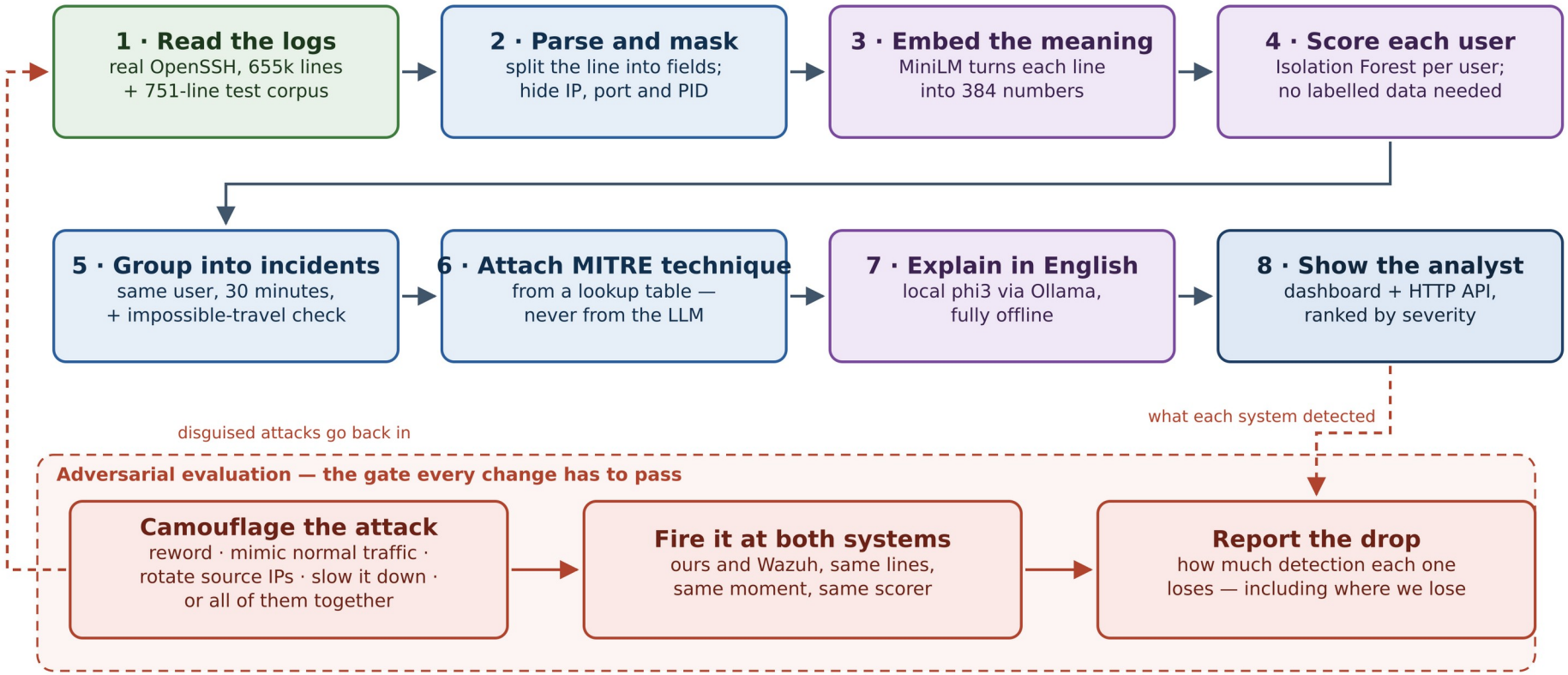
What the literature does	**Gap we work in**
Rule-based SIEMs match fixed text patterns [R2, R7]	They miss the same attack once it is reworded — we measured 91% → 29%
Log anomaly models are evaluated on clean benchmark data [R1, R3, R4, R5]	Nobody reports how much detection survives an attacker who is hiding
Adversarial evasion is studied for network intrusion detection [R8]	Not for log analysis, and never against a production SIEM side by side

Project Objectives

- Parse raw Linux authentication and system logs into a structured, maskable form.
- Detect anomalies from the meaning of a line, using sentence embeddings and a per-user Isolation Forest, with no labelled training data.
- Group related anomalies into incidents and attach the matching MITRE ATT&CK technique from a lookup table, never from the language model.
- Explain each incident in plain English with a local LLM that runs offline.
- Measure this system and a production SIEM (Wazuh) on the same logs under eight camouflage strategies, and report the difference — including where we lose.
- Keep the whole system running on a laptop with no GPU and no cloud service.

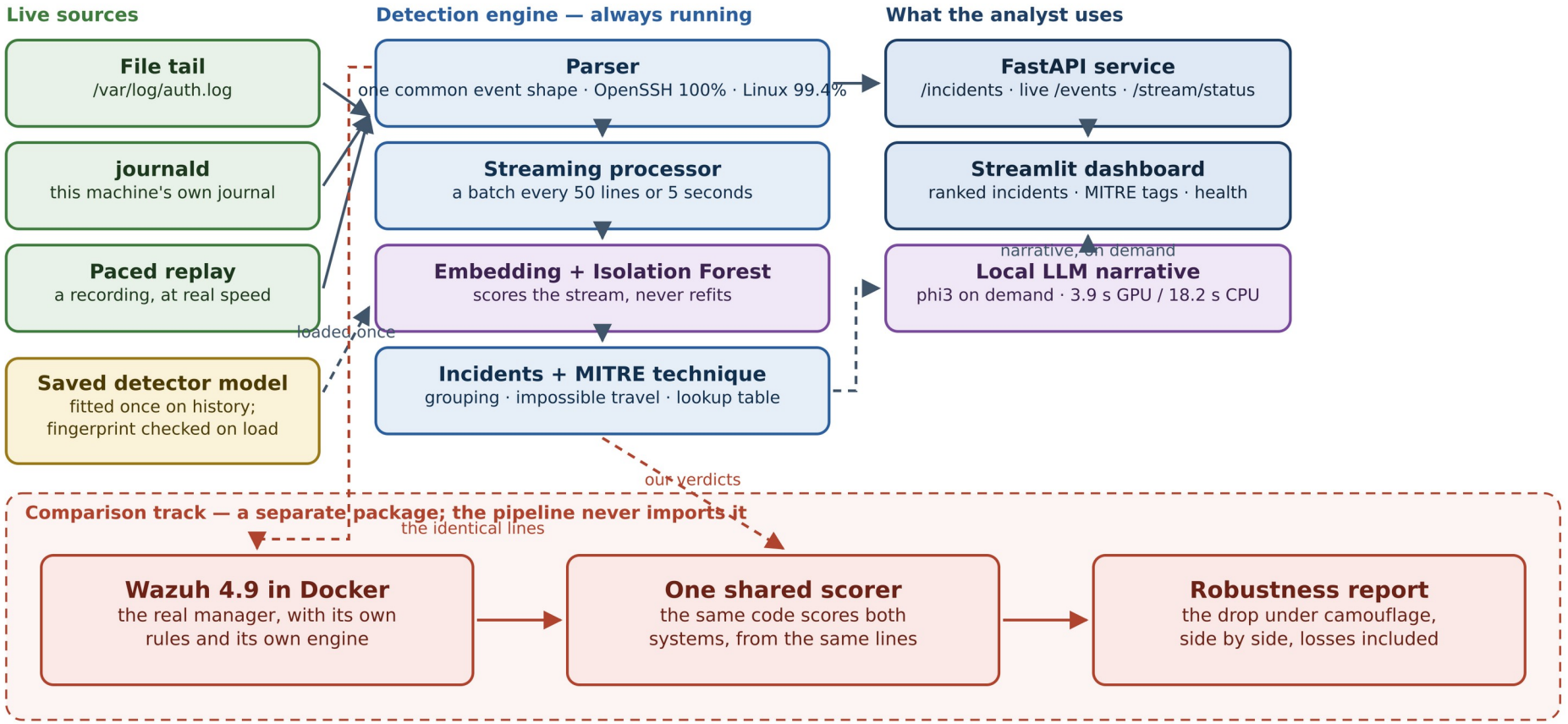
Proposed Methodology

- Read logs → parse and mask → embed the meaning → score each user → group into incidents → attach MITRE technique → explain in English → show the analyst.
- Camouflaged attacks are fed back in and both systems re-scored on every change.
- Isolation Forest over embeddings is standard; the measurement, not the detector, is what this project contributes.



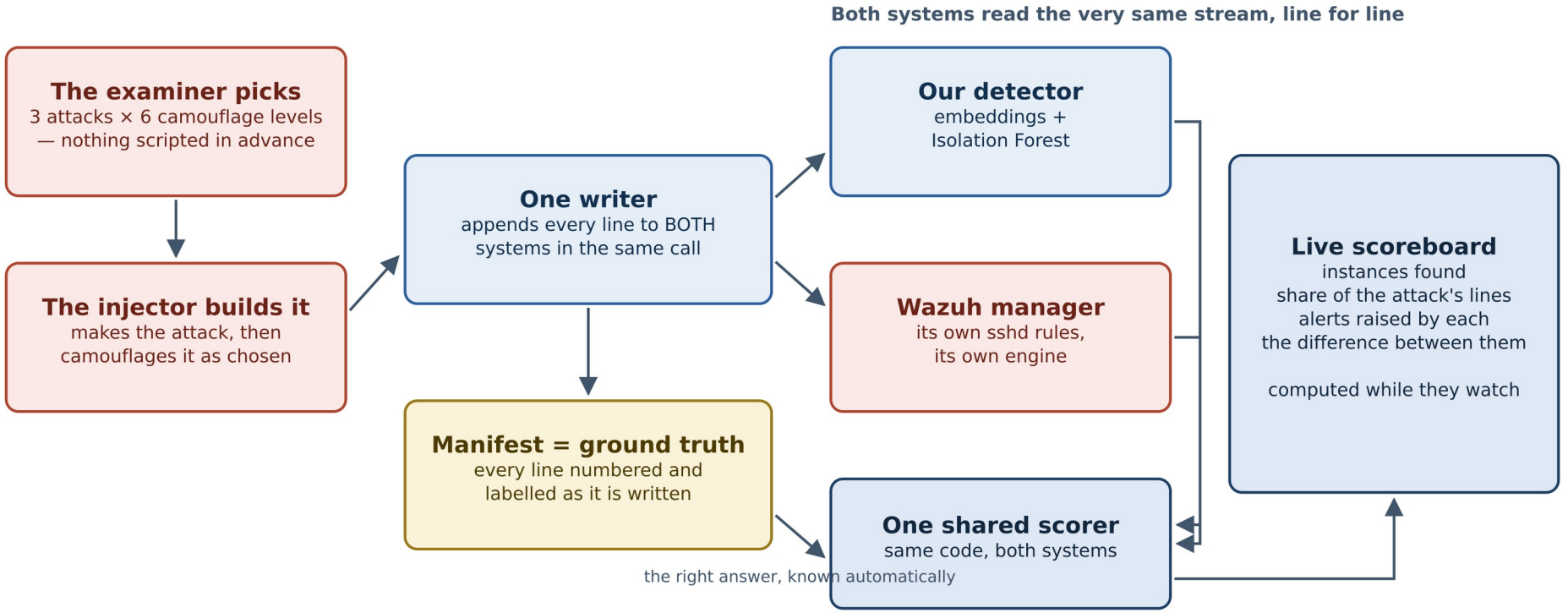
System Architecture / Flow Diagram

- Three live sources: a tailed file, this machine’s journal, or a paced replay.
- The model is fitted once on history and only scores the stream after that.
- Streaming reproduces the batch result exactly: 751 lines → 130 events → 94 incidents.
- The Wazuh comparison track is a separate package the pipeline may never import.



Implementation — Keeping the Comparison Fair

- One writer appends every line to both systems in the same call, so neither can be handed different traffic.
- Our injector wrote the attack, so every line's label is recorded as it goes out: ground truth is automatic.
- The same scoring code grades both systems.
- Built so far: foundation and 449 automated tests · benchmark platform with a real SIEM · live streaming, API and dashboard · the live bake-off.



Results — Robustness Under Camouflage

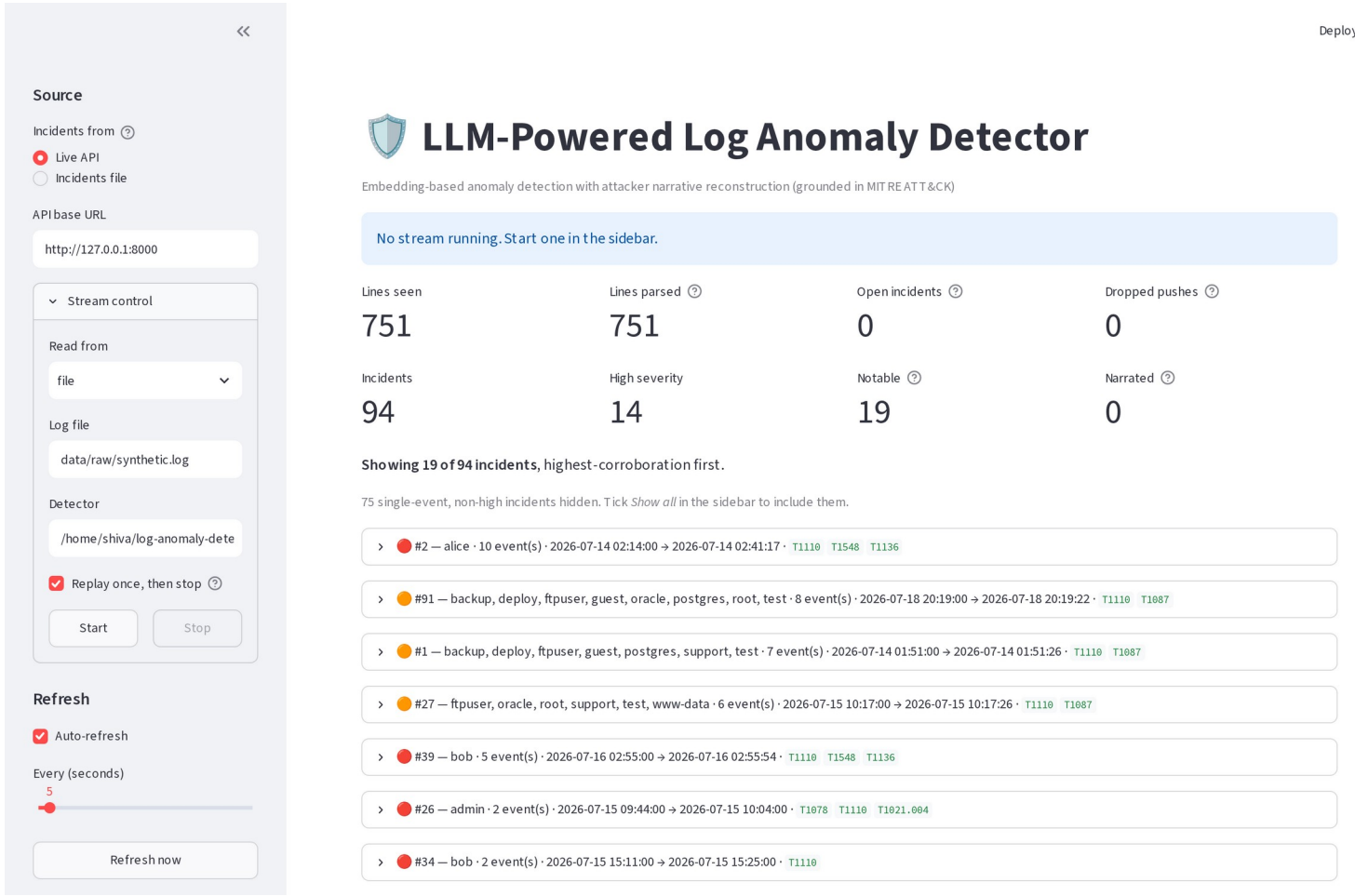
Drop in instance detection when the attack is camouflaged; bigger is worse. 5 seeds, real OpenSSH logs, both systems on the same corpus.

Camouflage strategy	**Our drop**	**Wazuh's drop**	**Difference**
Reworded (rephrase)	−2.2	−33.3	+31.1 in our favour
Mimicry / noise / volume / timing	0.0	0.0	tie × 4
IP rotation	−11.1	0.0	−11.1 against us
Slowed down	−8.9	0.0	−8.9 against us
Realistic combination	−13.3	0.0	−13.3 against us

Results — How to Read Them

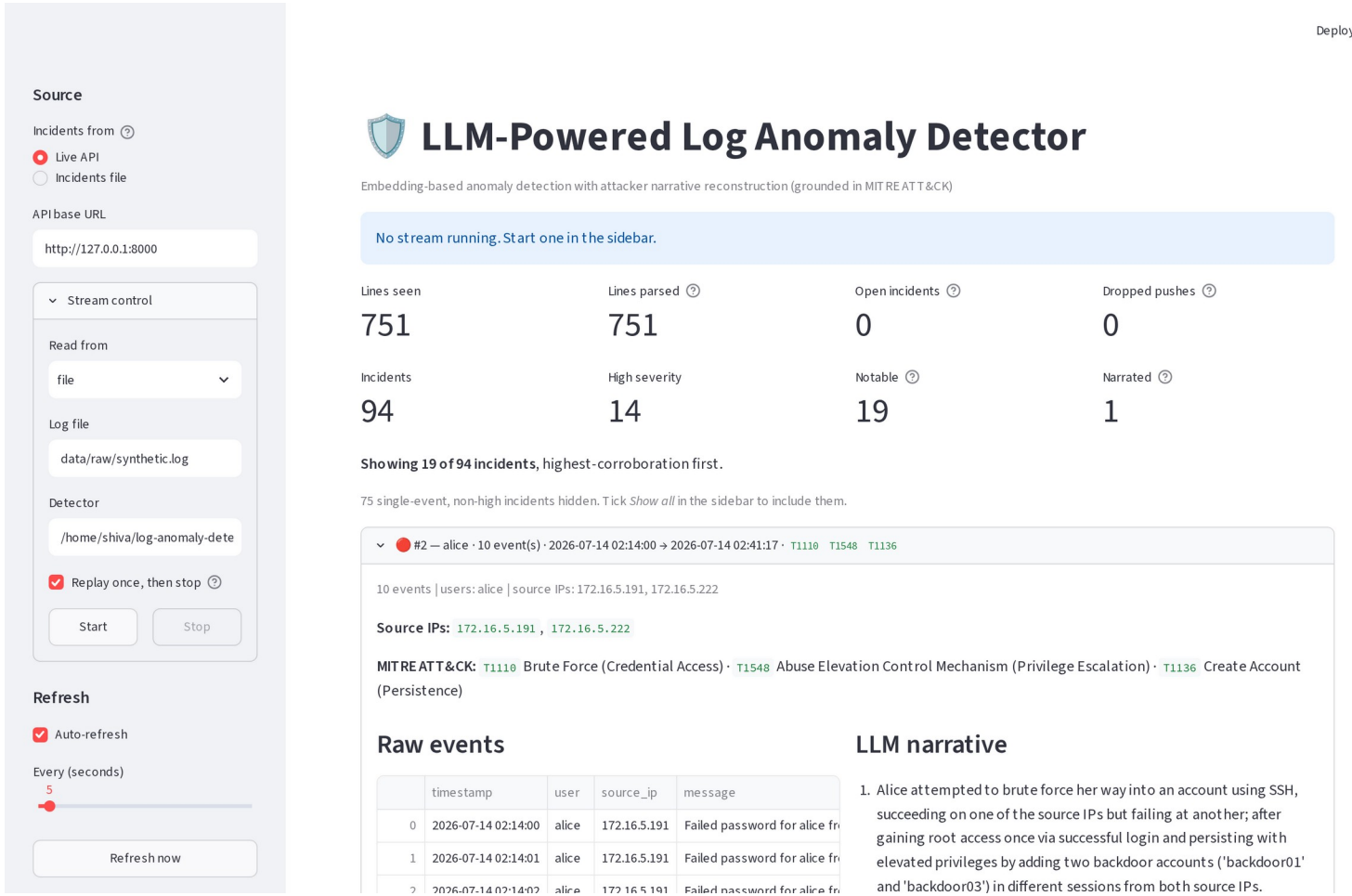
- 1 win, 4 ties, 3 losses, pooled over 5 seeds on real OpenSSH logs. The registered threshold (≥25 points on at least one strategy, written into git before any result existed) is cleared by rewording.
- Live: rewording an impossible-travel attack took Wazuh from 3 of 3 instances to 0 of 3, while we stayed at 3 of 3.
- Where we are weaker: precision 0.29 — 94 incidents for 9 real attacks; impossible travel is caught by a distance-over-time rule, not by the embeddings.

Working System (1 of 3) — Live Detection



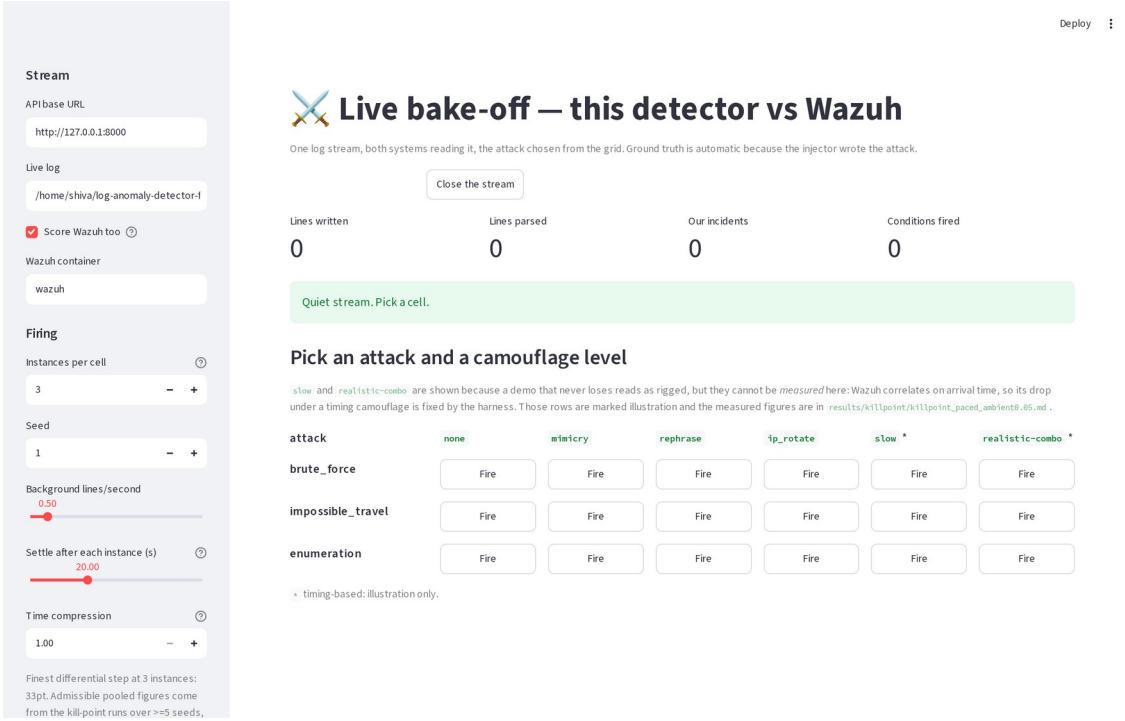
The dashboard reading the running service: 751 lines seen, 751 parsed, 94 incidents, 19 notable, with the stream's own health above the list.

Working System (2 of 3) — One Incident Explained

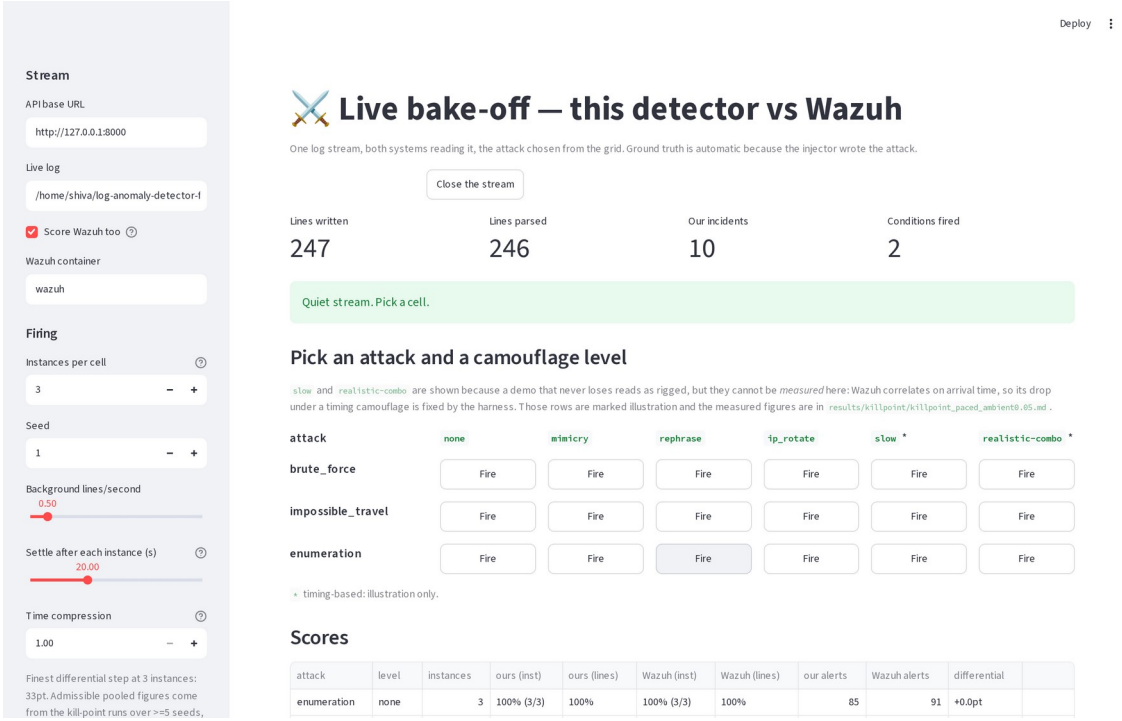


The events that made the incident, the source addresses, the MITRE techniques from the lookup table, and the narrative written by phi3 on this laptop.

Working System (3 of 3) — The Live Bake-Off



The bake-off grid — the examiner picks the attack and the camouflage level



Both systems scored live on the same firing — 3 of 3 instances each, 100% of lines

Technologies and Resources

- Software: Python 3.11 with uv · sentence-transformers (all-MiniLM-L6-v2) · scikit-learn Isolation Forest · Ollama + phi3 for the local narrative.
- Comparison: the real Wazuh 4.9 manager in Docker, running its own rules.
- Interfaces and testing: FastAPI service · Streamlit dashboard · 449 pytest tests · GitHub Actions on every push.
- Data: LogHub OpenSSH (655,147 real lines, contains real internet attacks), LogHub Linux (99.4% parsed), and a 751-line synthetic corpus with full ground truth.
- Hardware, minimum and fully supported: any laptop, no GPU, 8 GB RAM.
  - Optional NVIDIA GPU only speeds the written explanation up (3.9 s vs 18.2 s).
  - Demonstration setup: two laptops — one attacking and running the SIEM, one detecting.

Project Schedule & Current Progress

Task	Month 1	Month 2	Month 3	Month 4
Literature review	✓			
Requirement analysis	✓	✓		
Design		✓		
Development		✓		
Testing		✓		
Documentation		✓		

- Done so far: foundation and test suite · benchmark platform with a real SIEM · the pre-registered robustness measurement · live streaming, API and dashboard · the live side-by-side bake-off.
- Next: sequence modelling to fix precision, then an entity graph for correlation, then attacks generated by real tools.

References

- [1] C. Almodovar, F. Sabrina, S. Karimi and S. Azad, “LogFiT: Log anomaly detection using fine-tuned language models,” IEEE Trans. Network and Service Management, 21(2), pp. 1715–1723, 2024. doi:10.1109/TNSM.2024.3358730
- [2] M. Landauer, S. Onder, F. Skopik and M. Wurzenberger, “Deep learning for anomaly detection in log data: A survey,” Machine Learning with Applications, 12, 100470, 2023. doi:10.1016/j.mlwa.2023.100470
- [3] S. Ali, C. Boufaied, D. Bianculli, P. Branco and L. Briand, “A comprehensive study of machine learning techniques for log-based anomaly detection,” Empirical Software Engineering, 30(5), 129, 2025. doi:10.1007/s10664-025-10669-3
- [4] X. Wu, H. Li and F. Khomh, “On the effectiveness of log representation for log-based anomaly detection,” Empirical Software Engineering, 28(6), 137, 2023. doi:10.1007/s10664-023-10364-1
- [5] Z. Khan, D. Shin, D. Bianculli and L. Briand, “Impact of log parsing on deep learning-based anomaly detection,” Empirical Software Engineering, 29(6), 139, 2024. doi:10.1007/s10664-024-10533-w
- [6] G. Pang, C. Shen, L. Cao and A. van den Hengel, “Deep learning for anomaly detection: A review,” ACM Computing Surveys, 54(2), pp. 1–38, 2021. doi:10.1145/3439950
- [7] H. Kheddar, “Transformers and large language models for efficient intrusion detection systems: A comprehensive survey,” Information Fusion, 124, 103347, 2025. doi:10.1016/j.inffus.2025.103347
- [8] I. Debicha, R. Bauwens, T. Debatty, J.-M. Dricot, T. Kenaza and W. Mees, “TAD: Transfer learning-based multi-adversarial detection of evasion attacks against network intrusion detection systems,” Future Generation Computer Systems, 138, pp. 185–197, 2023. doi:10.1016/j.future.2022.08.011

Thank You !!!

Q & A